

Autoload e namespace

Autoload e namespace

Come organizzare file, classi e nomi in progetti PHP più grandi.

Il problema dei file

In un progetto piccolo puoi usare `require` per includere file a mano. In un progetto più grande diventa scomodo.

L'autoload carica le classi quando servono, senza scrivere `require` per ogni file.

Namespace

Un namespace è uno spazio dei nomi. Serve a evitare conflitti tra classi con lo stesso nome.

```
<?php
namespace App\Model;

class Prodotto {
}
```

Questa classe si chiama davvero `App\Model\Prodotto`.

use

Per usare una classe con namespace puoi importarla con `use`.

```
<?php
use App\Model\Prodotto;

$prodotto = new Prodotto();
```

Senza `use` , dovresti scrivere il nome completo.

Struttura delle cartelle

Una struttura comune e:

```
progetto/
  src/
    Model/
      Prodotto.php
  vendor/
  composer.json
```

Il file `Prodotto.php` contiene la classe `App\Model\Prodotto` .

Autoload con Composer

Nel `composer.json` puoi configurare l'autoload:

```
{
  "autoload": {
    "psr-4": {
      "App\\": "src/"
    }
  }
}
```

Poi esegui:

```
composer dump-autoload
```

Nel codice principale carichi l'autoload:

```
<?php  
require __DIR__ . '/vendor/autoload.php';
```

Nomi coerenti

Usa nomi di classi chiari e una cartella coerente con il namespace. Questo rende il progetto più facile da navigare.

Un file, una classe principale

Una convenzione semplice è mettere una classe principale per file.

```
<?php  
namespace App\Model;  
  
class Prodotto {  
}
```

Questo file potrebbe chiamarsi `src/Model/Prodotto.php`. Il percorso rispecchia il namespace.

Perche serve dump-autoload

Quando cambi la configurazione dell'autoload in `composer.json`, Composer deve rigenerare le informazioni interne.

```
composer dump-autoload
```

Se aggiungi una nuova classe seguendo la struttura già configurata, spesso non serve fare altro. Se invece cambi la mappa dei namespace, esegui questo comando.

Errore comune: namespace e cartella incoerenti

Se dichiari:

```
<?php  
namespace App\Model;
```

ma metti il file in una cartella non collegata ad `App\\`, Composer potrebbe non trovarlo. Quando una classe non viene caricata, controlla nome del namespace, nome della classe, percorso del file e configurazione `psr-4`.